

Lec 09

Application layer: FTP & E-mail

Computer Networks (CSE 232)

Rinku Shah

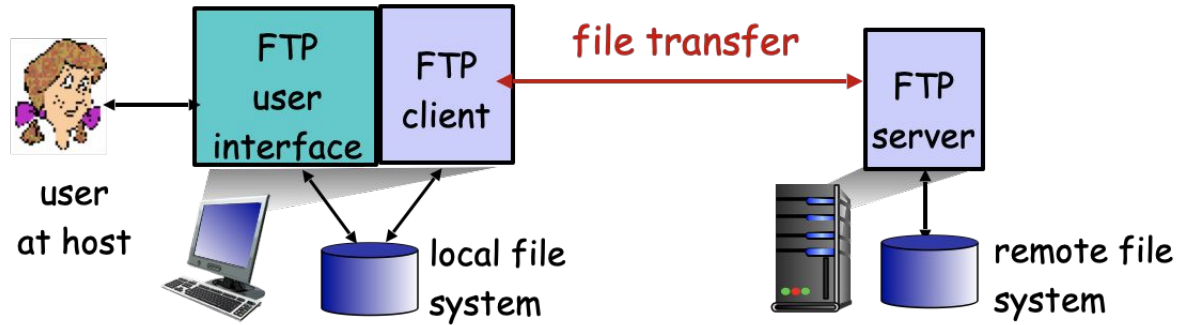
These slides are inspired and adapted from:

1. *Computer Networking: A Top-Down Approach*
6th edition, Jim Kurose, Keith Ross, Pearson, 2017



INDRAPRASTHA INSTITUTE of
INFORMATION TECHNOLOGY
DELHI

FTP: File Transfer Protocol



- Transfer file to/from remote host
- Uses client/server model
 - **client:** initiates transfer
 - **server:** remote host

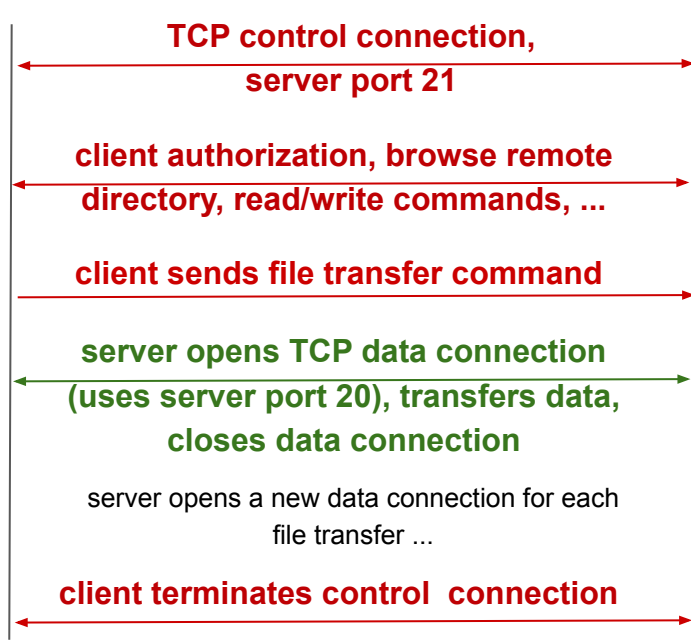
FTP: separate data and control connections



FTP client



FTP server



- **Connection-oriented protocol**
 - runs over TCP
- **TCP control connection**
 - Uses server port 21
 - Control connection is “Out of band”
- **TCP data connection**
 - Uses server port 20
- **Stateful protocol**
 - FTP server maintains state
 - authentication state
 - current directory

FTP commands, responses

sample commands

- sent as ASCII text over control channel
- **USER *username***
- **PASS *password***
- **LIST** return list of file in current directory
- **RETR *filename*** retrieves (gets) file
- **STOR *filename*** stores (puts) file onto remote host

sample return codes

- status code and phrase (as in HTTP)
- **331 Username OK, password required**
- **125 data connection already open; transfer starting**
- **425 Can't open data connection**
- **452 Error writing file**

Electronic Mail (E-mail)

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agents

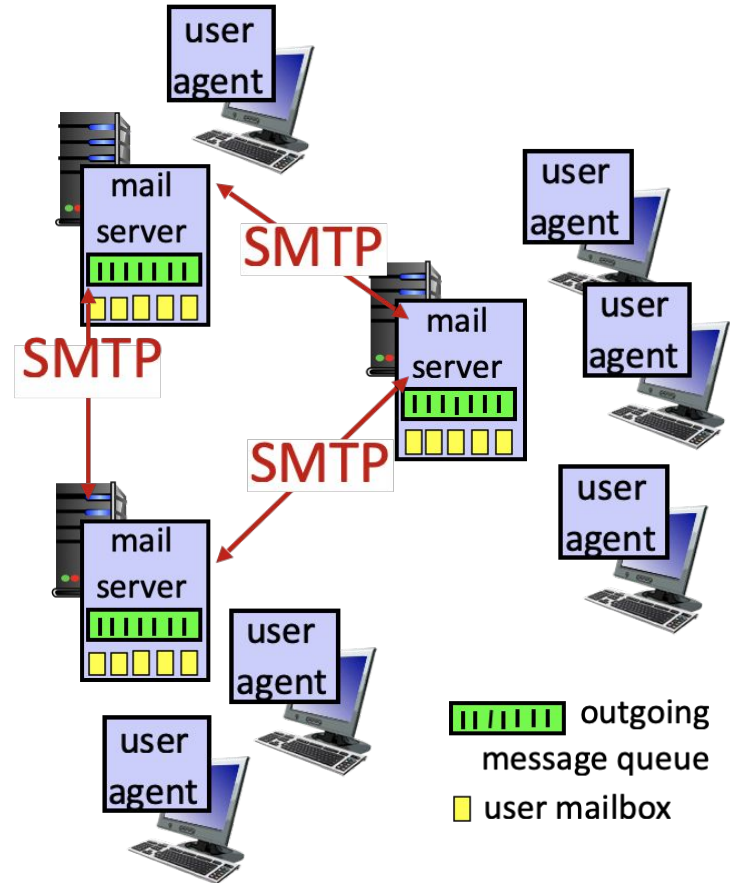
- composing, editing, reading mail messages
- e.g., Outlook

Mail servers

- outgoing, incoming messages are stored here
- **Mailbox**: contains incoming messages for user
- **Message queue**: outgoing mail messages

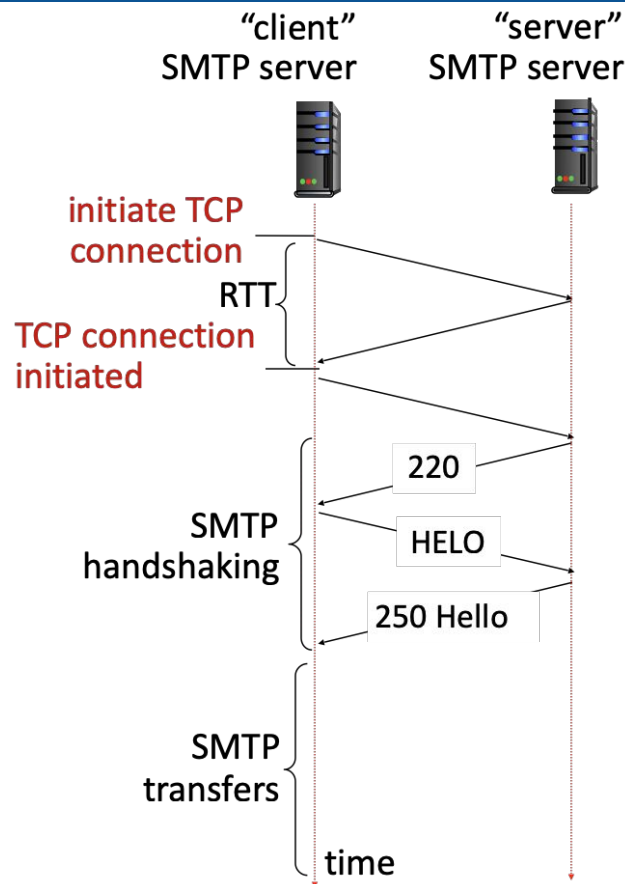
Simple Mail Transfer Protocol (SMTP)

- **Protocol between mail servers**
 - a. client: sending mail server
 - b. server: receiving mail server

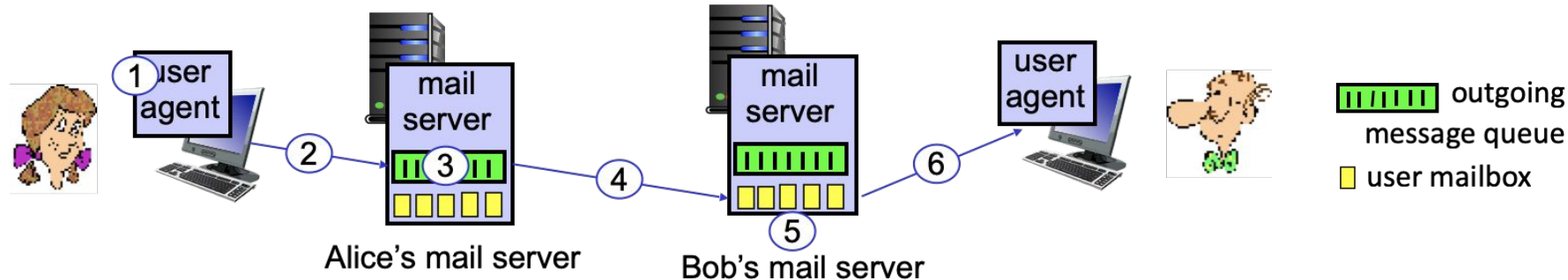


SMTP (RFC 5321)

- uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
 - direct transfer: sending server (acting like client) to receiving server
- three phases of transfer
 - SMTP handshaking (greeting)
 - SMTP transfer of messages
 - SMTP closure
- command/response interaction (like HTTP)
 - **commands**: ASCII text
 - **response**: status code and phrase



Scenario: Alice sends e-mail to Bob

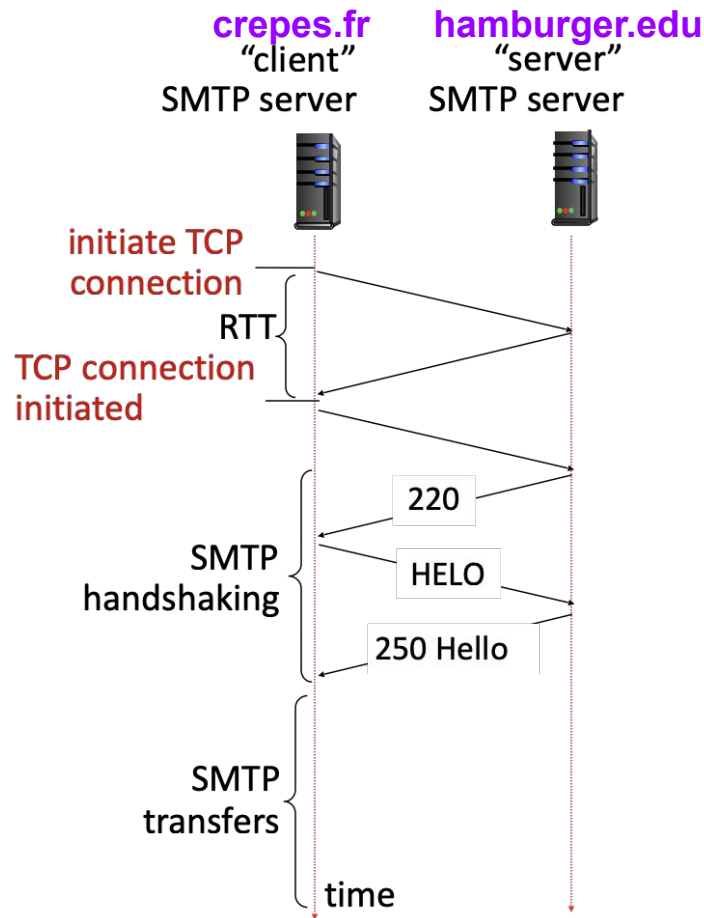


- 1) Alice uses UA to compose e-mail message "to" bob@someschool.edu
- 2) Alice's UA sends message to her mail server using SMTP; message placed in message queue
- 3) client side of SMTP at mail server opens TCP connection with Bob's mail server

- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message

Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```



SMTP compared with HTTP

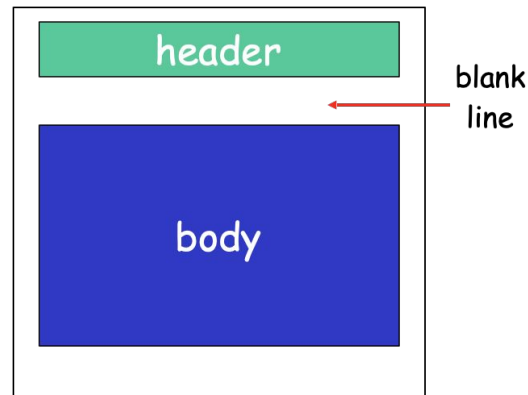
HTTP	SMTP
Client PULL	Client PUSH
Each object encapsulated in its own response message	Multiple objects sent in multipart message
Both have ASCII command/response interaction & status codes	

SMTP

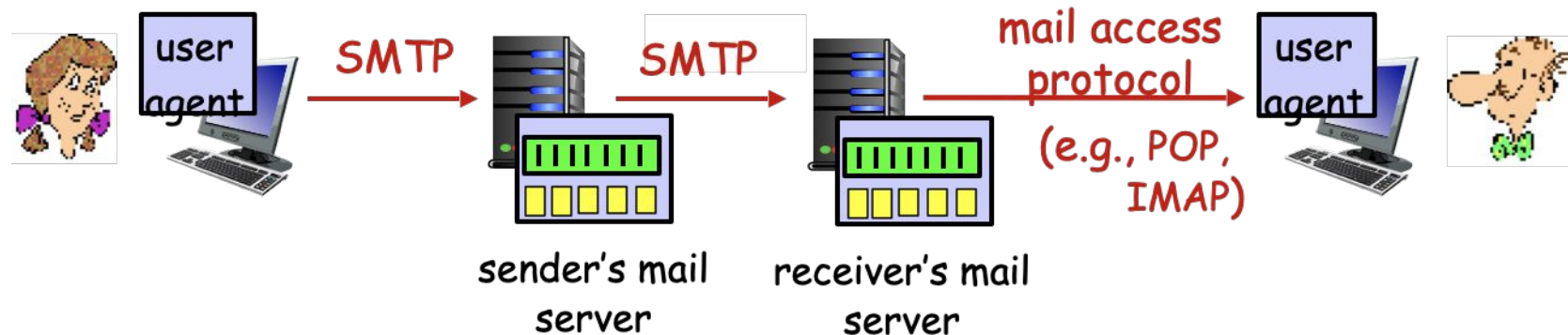
- Uses ***persistent connections***
- Requires message (header & body) to be in 7-bit ASCII
- SMTP server uses **CRLF.CRLF** to determine end of message

Mail message format

- SMTP: protocol for exchanging email msgs
- RFC 822
 - standard for text message format
- header lines, e.g.,
To:
From:
Subject:
different from SMTP MAIL FROM, RCPT TO: commands!
- Body: the “message”
 - ASCII characters only



Mail access protocols



- **SMTP**: delivery/storage to receiver's server
- Mail access protocol: retrieval from server
 - **POP**: Post Office Protocol [RFC 1939]
 - authorization, download
 - **IMAP**: Internet Mail Access Protocol [RFC 3501]
 - messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
 - **HTTP**
 - Gmail, Hotmail, Yahoo! Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

POP3 protocol

authorization phase

- ❖ client commands:
 - **user**: declare username
 - **pass**: password
- ❖ server responses
 - **+OK**
 - **-ERR**

transaction phase, client:

- ❖ **list**: list message numbers
- ❖ **retr**: retrieve message by number
- ❖ **dele**: delete
- ❖ **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

more about POP3

- previous example uses POP3
“*download and delete*” mode
 - Bob cannot re-read e-mail if he changes client
- POP3 “*download-and-keep*”:
copies of messages on different clients
- POP3 is stateless across sessions

IMAP

- keeps all messages in one place:
at server
- allows user to organize messages
in folders
- keeps user state across sessions:
 - names of folders and
mappings between message
IDs and folder name